Particle System

HW1
2026 Computer Animation and Special Effects

Build & Exec

- Under the project folder
- Linux
  - cd build
  - cmake ..
  - make
  - ./make
- Windows
  - cd build
  - cmake ..
  - cmake --build ./ --config Release
  - ./Release/main.exe

Objective

- src
  - main.cpp
    - change your studentID
  - jelly.cpp
    - Eigen::Vector3f Jelly::computeSpringForce(...)
    - Eigen::Vector3f Jelly::computeDamperForce(...)
    - void Jelly::computeInternalForce()
  - terrain.cpp
    - void PlaneTerrain::handleCollision(...)
  - integrator.cpp
    - void ExplicitEulerIntegrator::integrate(...)
    - void ImplicitEulerIntegrator::integrate(...)
    - void PositionBasedDynamicIntegrator::integrate(...)
    - void RungeKuttaFourthIntegrator::integrate(...)

Objective (cont.)

- void Jelly::computeInternalForce()
  - Trace every spring and apply the force accordingly
    - Eigen:: Jelly::computeSpringForce()
      - compute spring forces
    - Eigen:: Jelly::computeDamperForce()
      - compute damper forces
  - The values of parameter springCoef and damperCoef are defined in massSpringSystem.cpp
  - Hint: review “particles.pptx” from p.9 - p.13
- void PlaneTerrain::handleCollision(...)
  - Handle collision between plane and jelly
  - Compute the velocity after collision
  - Hint: review “particles.pptx” from p.14 - p.19

Objective (cont.)

- Integrator
  - Update particles' position and velocity
  - void ExplicitEulerIntegrator::integrate(...)
    - Hint: review “ODE_basics.pptx” from p.15 - p.16
  - void ImplicitEulerIntegrator::integrate(...)
    - Hint: review "ODE_implicit.pptx" from p.18 - p.19
  - void PositionBasedDynamicIntegrator::integrate(...)
    - Hint: https://matthias-research.github.io/pages/publications/posBasedDyn.pdf
- Bonus
  - void RungeKuttaFourthIntegrator::integrate(...)
    - Hint: review “ODE_basics.pptx” p.21

Position-Based Dynamic

The PBD pipeline usually follows these steps:

1. Apply external forces
2. Predict positions
3. Solve constraints
4. Update velocities
5. Write final positions

PBD - Step 1: Prediction Step

- **Identity to explicit Euler method**
- Hint: use setPredictedPosition instead of setPosition

PBD - Step 2: Solve constraints

- **Terrain Constraint:**
  - Similar to self collision
- **Self Collision**

- minDistance and maxDistance are depending on the jelly cell size
- If particles become too close (dist < minDistance) or to far away to their neighborhood then:

  - **Update the position of both particles symmetrically**
    - If too close -> push
    - If too far -> pull
    - Update the particle and its neighborhood
    - The distance you push/pull should according to the distance to the neighborhood

- Hint: use setPredictedPosition instead of setPosition

PBD - Step 3: Position and Velocity Update with XSPH Velocity

1. Update the velocity according to the constrained position, remember to update the constrained position to the position
2. Apply XSPH:

   iterate over all particles p
   - weight_sum=0; velocity_correction=0
   - iterate over all neighborhood q
     - w = 1 / distance(p, q)
     - weight_sum += w
     - velocity_correction += w * (q.velocity - p.velocity)
   - p.velocity += xsphFactor * velocity_correction / weight_sum

Scoring

- Change window title to “Soft-body Simulation **STUDENT_ID**” (0%)
  - -10% if title is wrong
- Compute internal forces - 10%
- Handle ground Collision - 20%
- Integrator - 50%
  - Explicit Euler - 5%
  - Implicit Euler - 5%
  - Position-Based Dynamic - 40%
- Report - 20%
- Bonus - Runge-Kutta 4th - 15%

Report & Video

- Suggested outline
  - Implementation
  - Result and Discussion
    - The difference between integrators
    - Effect of parameters (springCoef, damperCoef, coefResist, coefFriction, etc.)
  - Bonus (Optional)
  - Conclusion
- Video
  - A short video to demo your result

Submission

- Please upload hw1_<your student ID>.zip
- hw1_<your student ID> <- compress this folder
  - src
  - main.cpp
  - video_<your student ID>.mp4
  - softsim_<your student ID>.exe for **Windows user** or softsim_<your student ID> for **Linux user**
  - report_<your student ID>.pdf
- After unzip your upload file it's should be a folder called hw1_<your student ID>
- Late policies
  - Penalty of 10 points on each day after deadline
- Cheating policies
  - 0 points for any cheating on assignments
- Deadline
  - Monday, 2026/04/06, 23:59

Note

- Read TODOs in the template and follow TODOs' order

```
// TODO#1: Connect particles with springs.
//   1. Consider the type of springs and compute indices of particles which the spring connect to.
//   2. Compute rest spring length using particle positions.
//   3. Iterate the particles. Push spring objects into `springs` vector
// Note:
//   1. The particles index can be computed in a similar way as below:
//   ==========================================
//   0 1 2 3 ... particlesPerEdge
//   particlesPerEdge + 1 ....
//   ... ... particlesPerEdge * particlesPerEdge - 1
//   ==========================================
// Here is a simple example which connects the structrual springs along z-axis.
```

- How to contact TAs?
  - please ask your questions on new E3 forum or send email to ALL TAs via new E3
  - if you need to ask questions face-to-face, please send an email for appointment